



La concepción inicial de los sistemas lleva consigo el desarrollo de diagramas en procesos estructurales, posteriormente en el adelanto de las nuevas tendencias surge el desarrollo de modelos de negocio con el diseño orientado a objetos; consecutivamente ésta vertiente da paso a un nuevo lenguaje de modelado que combina varios métodos y creado por varios autores – James Rumbaugh, Ivar Jacobson, Grady Booch – llamado UML (Unified Model Language); ésta, es una de las herramientas más novedosas en el modelado de sistemas de información, permitiendo a los modeladores capturar el comportamiento de los sistemas de un modo convencional y sencillo de comprender por el resto de las personas.

A modo ilustrado, se tratarán temas referentes al lenguaje UML, su uso, conceptos y modelos en el desarrollo de sistemas.

1. Para qué sirve UML

Antes de indicar para qué sirve UML, es importante definirlo.

Según un artículo publicado por Vico Open Modeling, S.L. (2008), el UML “Es un lenguaje visual orientado al modelado de sistemas”.

Igualmente en Wikipedia, la enciclopedia libre, UML “Es el lenguaje de modelado de sistemas de software más conocido y utilizado en la actualidad”.

El desarrollo de UML comenzó a finales de 1994 cuando Grady Booch y Jim Rumbaugh de Rational Software Corporation empezaron a unificar sus métodos. A finales de 1995, Ivar Jacobson y su compañía Objectory se incorporaron a Rational en su unificación, aportando el método OOSE (OOSE: Object- Oriented Software Engineering). En el proceso de creación de UML han participado, no obstante, otras empresas de gran peso en la industria como Microsoft, Hewlett-Packard, Oracle o IBM, así como grupos de analistas y desarrolladores.

El lenguaje modelado no es más que hacer una representación abstracta de un sistema físico con un propósito determinado, con el modelo se pretende capturar las partes esenciales del sistema.

Es importante resaltar que UML es un “lenguaje” para especificar y no para describir métodos o procesos. Se utiliza para definir un sistema de software, para detallar los artefactos en el sistema y para documentar y construir. En otras palabras, es el lenguaje en el que está descrito el modelo. Se puede aplicar en una gran variedad de formas para dar soporte a una metodología de desarrollo de software (tal como el Proceso Unificado Racional o RUP), pero no especifica en sí mismo qué metodología o proceso a usar.

¿Para qué sirve el UML?

El UML sirve para definir un problema que afecta la organización y plantear una solución de diseño, modelando los procesos involucrados.

Sirve para mejorar el nivel de comunicación formal dentro de un proyecto.

Sirve para representar visualmente las reglas de creación, estructura y comportamiento de un grupo relacionado de objetos y procesos.

Sirve para visualizar de manera eficiente la complejidad de un sistema o una organización en un reducido número de diagramas; es decir, abordar la complejidad con una documentación minimalista.

Sirve para mantener más ágilmente las especificaciones ante los cambios y nuevos enfoques de la arquitectura.

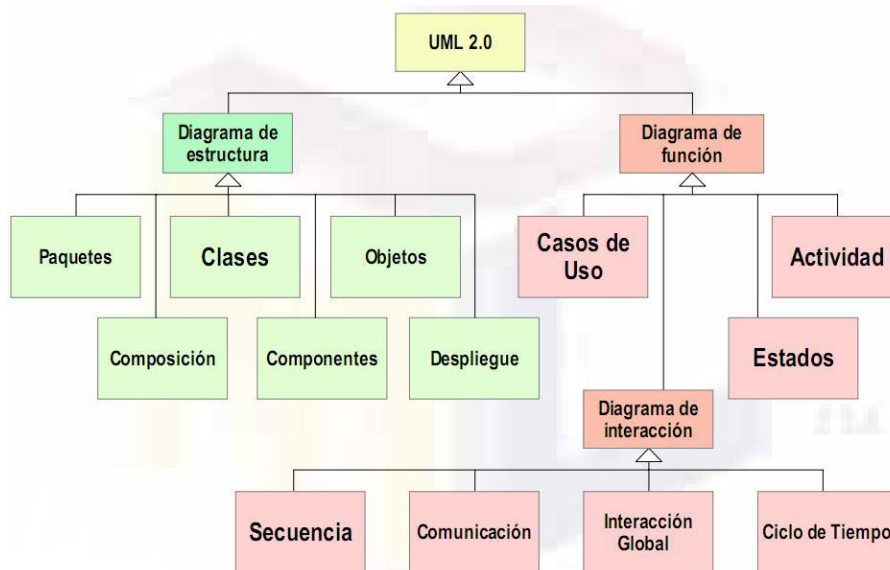
Sirve para desarrollar procesos/productos con una mayor fiabilidad y calidad; siendo el impacto de las decisiones más visibles.

Sirve para definir, organizar y compartir conocimientos.

2. Modelos o Diagramas qué contiene. Descripción. Características. Ejemplos.

Una vez que se tiene identificado el problema de un sistema, es indispensable la creación de un modelo que permita comprender el comportamiento del sistema antes de desarrollarlo. Todo modelo contiene detalles según el análisis previo que se hace del sistema. UML no solo permite la creación de un modelo sino que recomienda la utilización de varios modelos con los que se pueda obtener desde diferentes puntos de vista detalles que asemejen el diseño mucho más a la realidad.

Según Josep Vilalta Marzo (2008), la última versión de UML es la 2.0, y en ella hay 13 tipos de diagramas diferentes que se clasifican de la siguiente manera:



Fuente: Josep Vilalta, 2008. <http://vico.org>. Citado 15/02/2009

- Los **Diagramas de Estructura** enfatizan en los elementos que deben **existir** en el sistema modelado:
 - Diagrama de clases
 - Diagrama de objetos.
 - Diagrama de composición o Estructura compuesta.
 - Diagrama de componentes.
 - Diagrama de despliegue.
 - Diagrama de paquetes.
- Los **Diagramas de Funcionamiento** enfatizan en lo que debe **suced**er en el sistema modelado:
 - Diagrama de actividades.
 - Diagrama de casos de uso.
 - Diagrama de estados.

- Los **Diagramas de Interacción** son un subtipo de diagramas de comportamiento, que enfatiza sobre el **flujo de control y de datos** entre los elementos del sistema modelado:
 - Diagrama de secuencia.
 - Diagrama de colaboración.
 - Diagrama de ciclo de tiempo.
 - Diagrama de interacción global.

Los Diagramas de Estructura

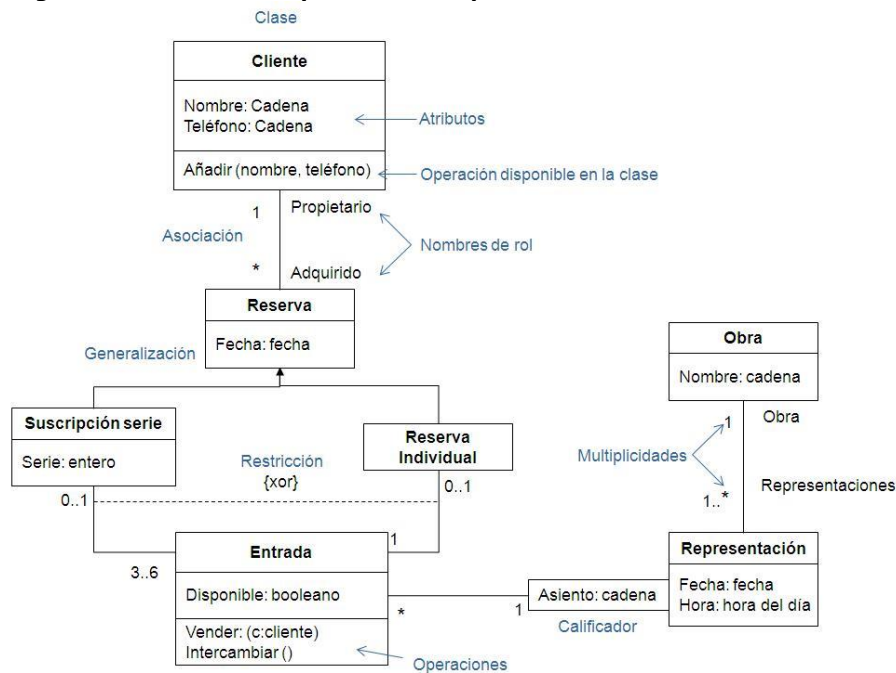
2.1 Diagramas de Clases: Las clases son el centro alrededor del cual se organiza la vista de clases; otros elementos pertenecen o se unen a las clases. Un diagrama de clases sirve para visualizar las relaciones entre las clases que involucran el sistema, las cuales pueden ser asociativas, de herencia, de uso y de contenimiento.

Características:

- Las clases se dibujan como rectángulos, mostrando el listado del nombre de la clase, atributos y operaciones en compartimientos separados.
- Las relaciones entre clases se dibujan como las líneas que conectan rectángulos de clases y existen tres relaciones la dependencia, generalización y asociación.
- Cada representación tiene muchas entradas disponibles, cada una con un número de asiento único.

Ejemplo:

Diagrama de Clases de la aplicación de taquilla



Fuente: Rumbaugh, James; Jacobson Ivar y Booch GRady, 2000

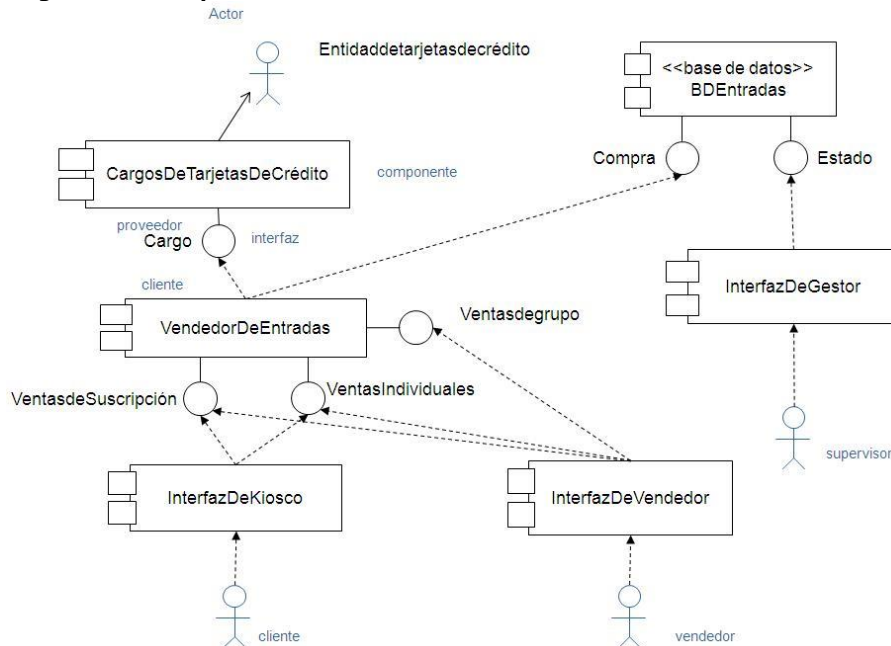
2.2 Diagrama de componentes: Un diagrama de componentes muestra dependencias entre los componentes – *un componente es una unidad física de implementación con interfaces bien definidas pensada para ser utilizada como parte reemplazable del sistema* – cada componente ofrece algunas interfaces y utiliza otras. Si las dependencias entre componentes se hacen a través de interfaces, los componentes se pueden sustituir por otros componentes que realicen las mismas interfaces.

Características:

- Muestra la organización y las dependencias entre un conjunto de componentes. No es necesario que un diagrama incluya todos los componentes del sistema, normalmente se realizan por partes. Cada diagrama describe un apartado del sistema.
- Un diagrama de componentes se representa como un grafo de componentes software unidos por medio de relaciones de dependencia (generalmente de compilación). Puede mostrar también que un componente software contiene una interfaz, es decir, la soporta.
- Normalmente los diagramas de componentes se utilizan para modelar código fuente, versiones ejecutables, bases de datos físicas.

Ejemplo:

Diagrama de Componentes



Fuente: Rumbaugh, James; Jacobson Ivar y Booch GRady, 2000

2.3 Diagrama de objeto:

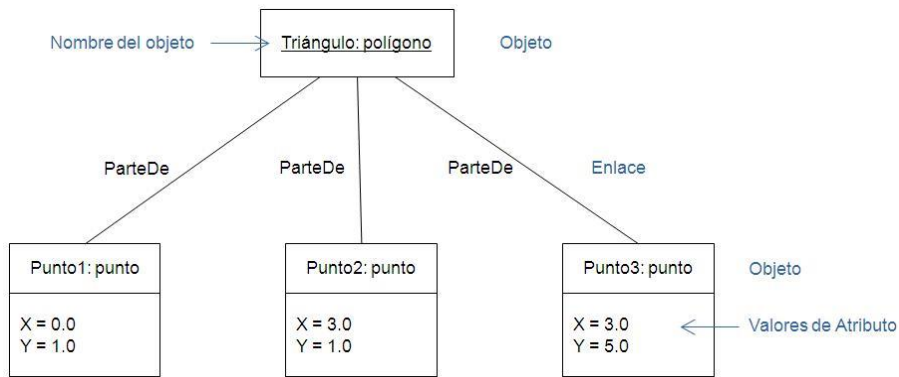
Los diagramas de objetos modelan las instancias de elementos contenidos en los diagramas de clases. Un diagrama de objetos muestra un conjunto de objetos y sus relaciones en un momento concreto.

Características:

- Los diagramas de objetos utilizan un subconjunto de los elementos de un diagrama de clase.
- Los diagramas de objetos no muestran la multiplicidad ni los roles, aunque su notación es similar a los diagramas de clase.
- Se puede considerar un caso especial de un diagrama de clases en el que se muestran instancias específicas de clases (objetos) en un momento particular del sistema.

Ejemplo:

Diagrama de Objeto



Fuente: Rumbaugh, James; Jacobson Ivar y Booch GRady, 2000

2.4 Diagrama de estructura compuesta: Un diagrama de estructura compuesta es un diagrama que muestra la estructura interna de un clasificador, incluyendo sus puntos de interacción a otras partes del sistema. Esto muestra la configuración y relación de las partes que juntas realizan el comportamiento de clasificador contenido.

Los elementos de clase han sido descriptos en gran detalle en la sección en los diagramas de clase. Esta sección describe la forma en que las clases se pueden mostrar como elementos compuestos exponiendo interfaces y conteniendo puertos y partes.

Características:

- Una *estructura compuesta* es un conjunto de elementos interconectados que colaboran en tiempo de ejecución para lograr algún propósito. Cada elemento tiene algún *rol* definido en la colaboración

Ejemplo:

Diagrama de Estructura Compuesta



Fuente: Diagrama de Estructura Compuesta <http://www.liderdeproyecto.com/uml/uml012.html>. Citado el 15/02/2009

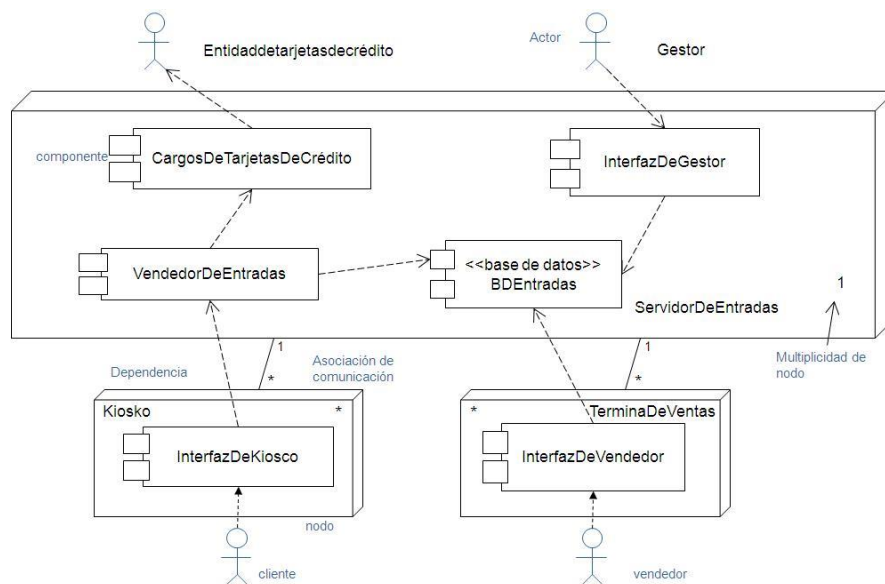
2.5 Diagrama de despliegue: Los diagramas de despliegues muestran las relaciones físicas entre los componentes físicos y lógicos del sistema final (hardware y software).

Características:

- Los elementos usados por este tipo de diagrama son nodos (representados como un prisma), componentes (representados como una caja rectangular con dos protuberancias del lado izquierdo) y asociaciones.
- El diagrama de despliegue, representa los artefactos del sistema como nodos, los cuales son conectados a través de caminos de comunicación para crear redes de sistemas de complejidad arbitraria.
- Describe la arquitectura en tiempo de ejecución de procesadores, dispositivos y los componentes de software que ejecutan ésta arquitectura.
- Describe una topología del sistema, estructura del hardware y el software que se ejecuta en cada unidad.

Ejemplo:

Diagrama de Despliegue



Fuente: Rumbaugh, James; Jacobson Ivar y Booch GRady, 2000

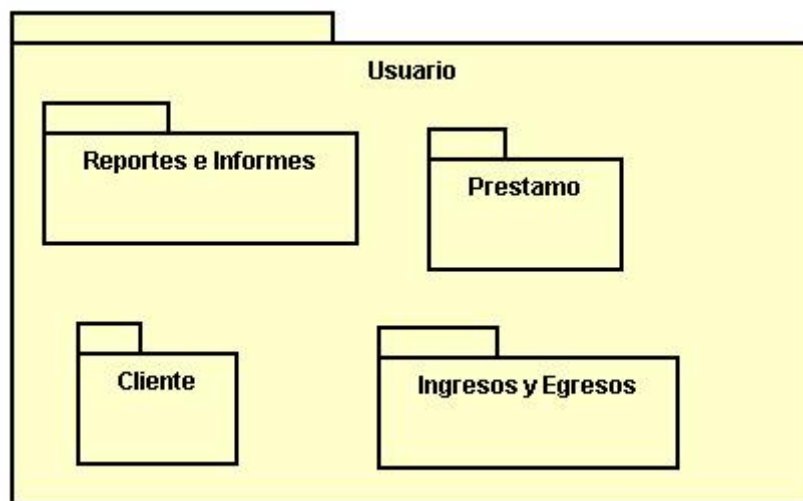
2.6 Diagrama de paquetes: Un diagrama de paquetes muestra como un sistema está dividido en agrupaciones lógicas mostrando las dependencias entre esas agrupaciones. Dado que normalmente un paquete está pensado como un directorio, los diagramas de paquetes suministran una descomposición de la jerarquía lógica de un sistema.

Características:

- Los paquetes son buenos elementos de gestión.
- Se usan paquetes en un modelo de desarrollo para agrupar elementos relacionados.
- Cada uno de los paquetes se pueden asignar a un individuo o a un equipo de desarrollo

Ejemplo:

Diagrama de Paquete



Fuente: Diagrama de paquete. <http://diario.de/osjobu/uml.html>. Citado 18/02/2009

Los Diagramas de Funcionamiento

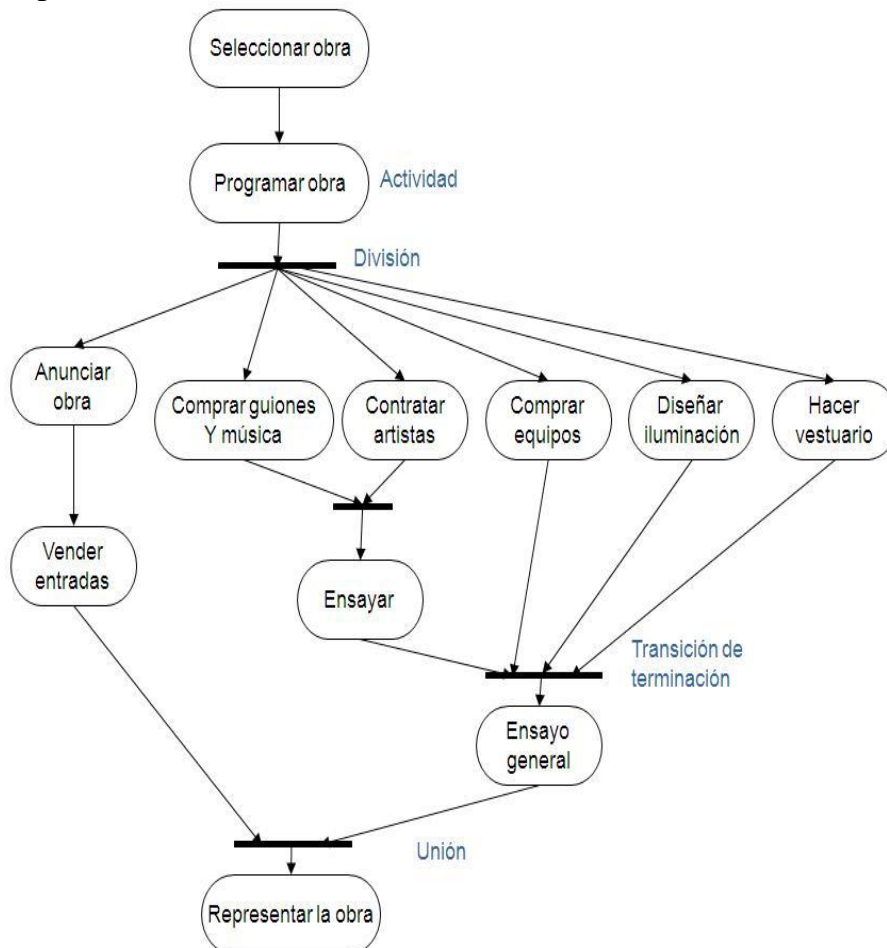
2.7 Diagrama de actividades: Este tipo de diagramas se pueden utilizar para modelar actividades de software, éste diagrama es provechoso para entender el comportamiento de alto nivel de la ejecución de un sistema, sin profundizar en los detalles internos de los mensajes.

Características:

- Un diagrama de actividades puede contener bifurcaciones, así como divisiones de control en hilos concurrentes.
- Un diagrama de actividad es la notación para un grafo de actividades, el cual es una forma especial de máquina de estados, prevista para modelar cómputos y flujos de trabajos.
- Los diagramas de actividad soportan actividades tanto secuenciales como paralelas. La ejecución paralela se representa por medio de iconos de fork/espera, y en el caso de las actividades paralelas, no importa en qué orden sean invocadas (pueden ser ejecutadas simultáneamente o una detrás de otra).
- Los diagramas de actividad siempre están asociados a una clase, a una operación o a un caso de uso.

Ejemplo:

Diagrama de Actividad



Fuente: Rumbaugh, James; Jacobson Ivar y Booch GRady, 2000

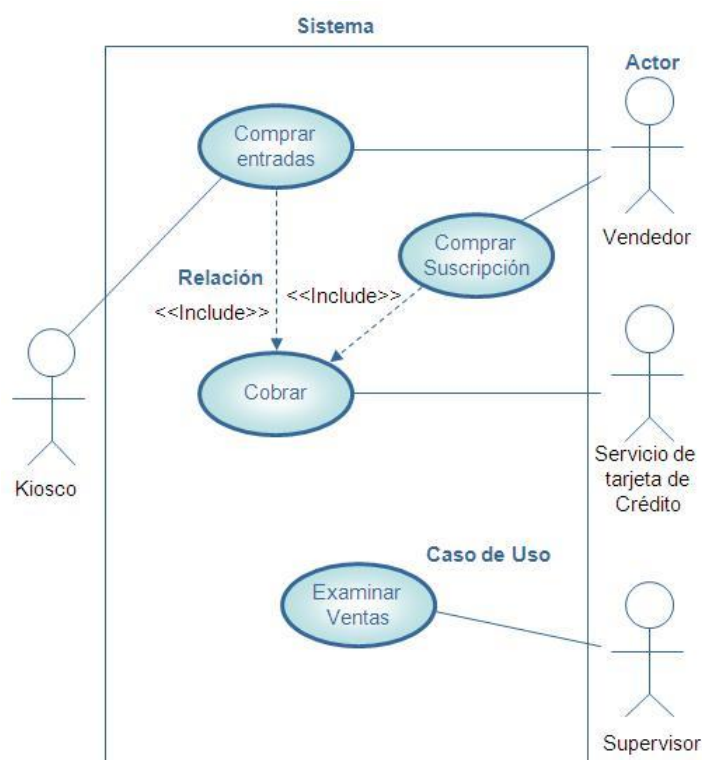
2.8 Diagrama de casos de uso: Un caso de uso es una unidad coherente de funcionalidad, expresada como transacción entre los actores y el sistema. El propósito de la vista de casos de uso es enumerar a los actores y los casos de uso, y demostrar que actores participan en cada caso de uso.

Características:

- La descripción escrita del comportamiento del sistema al afrontar una tarea de negocio o un requisito de negocio. Esta descripción se enfoca en el valor suministrado por el sistema a entidades externas tales como usuarios humanos u otros sistemas.
- La posición o contexto del caso de uso entre otros casos de uso. Dado que es un mecanismo de organización, un conjunto de casos de uso coherente, consistente promueve una imagen fácil del comportamiento del sistema, un entendimiento común entre el cliente/propietario/usuario y el equipo de desarrollo.

Ejemplo:

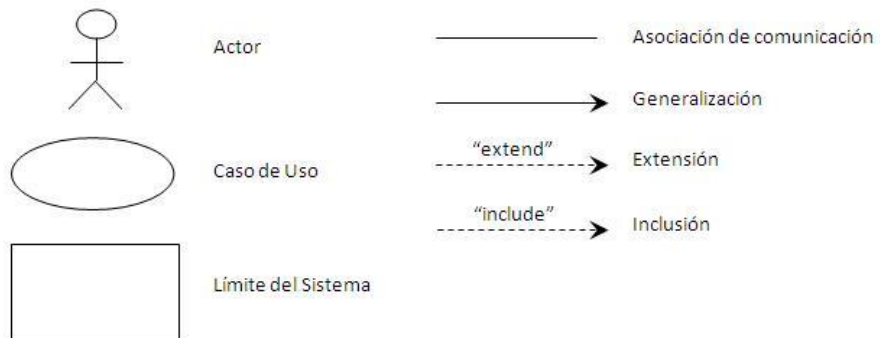
Diagrama de Casos de Uso de una taquilla



Fuente: Rumbaugh, James; Jacobson Ivar y Booch GRady, 2000

La simbología de los Diagramas de casos es:

Simbología de Diagrama de Casos de Usos



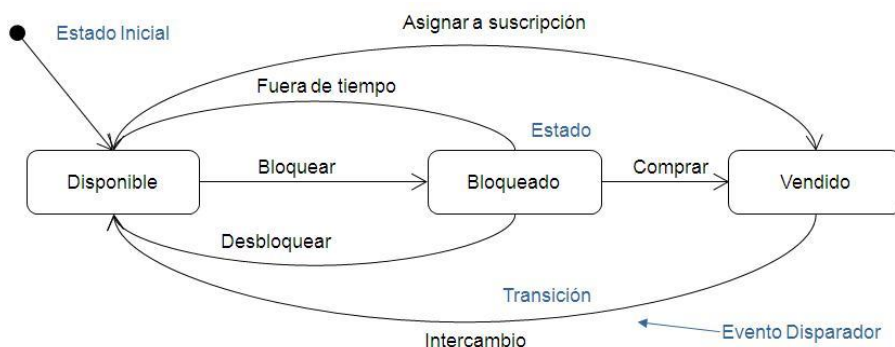
2.9 Diagrama de estado: Éste diagrama describe el comportamiento dinámico de los objetos, en un cierto plazo, modelando los ciclos de vida de los objetos de cada clase; tomando a cada objeto como una entidad aislada que se comunica con el resto del sistema a través de eventos. A su vez los eventos representan las clases de cambios por los que un objeto puede pasar.

Características:

- Un diagrama de estado describe el comportamiento de las clases, pero también el comportamiento dinámico de los casos de uso, de las colaboraciones, y de los métodos.
- Es un gráfico de estados y de transiciones. Los diagramas de estados se unen a una clase y describe, la respuesta de una instancia de la clase a los eventos que recibe.
- Los diagramas de estado son útiles para entender los mecanismos de control, tales como interfaces de usuarios o controladores de dispositivos.
- Define los estados que un objeto puede tener y como los eventos afectan esos estados.
- Captura el ciclo de vida de los objetos, subsistemas y sistemas.

Ejemplo:

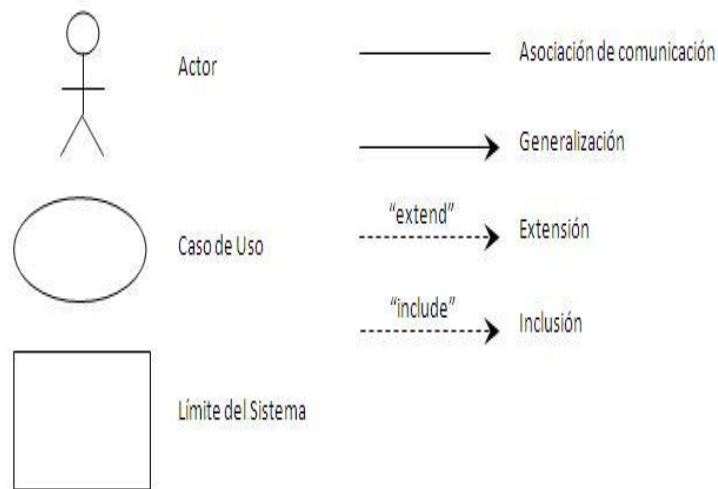
Diagrama de Estado



Fuente: Rumbaugh, James; Jacobson Ivar y Booch GRady, 2000

A continuación se presenta su simbología:

Simbología de Diagrama de Casos de Usos



Los Diagramas de Interacción

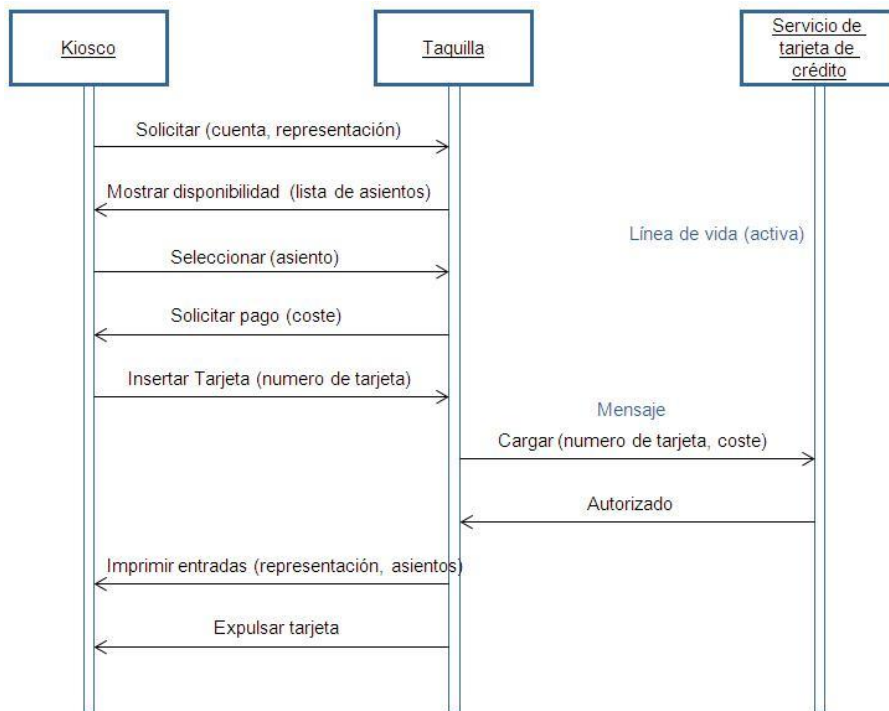
2.10 Diagrama de secuencia: Un diagrama de secuencia muestra un conjunto de mensajes, dispuestos en una secuencia temporal. Cada rol en la secuencia se muestra como una línea de vida, es decir, una línea vertical que representa el rol durante cierto plazo de tiempo, con la interacción completa. El diagrama de secuencia contiene detalles de implementación del escenario, incluyendo los objetos y clases que se usan para implementar el escenario, y mensajes pasados entre los objetos.

Características:

- Muestra la secuencia del comportamiento de un caso de uso.
- Muestra como los objetos interactúan entre ellos.
- Es específico para un escenario.
- Tiene dos ejes: un eje vertical que muestra el tiempo y un eje horizontal que muestra el grupo de objetos.
- Cada mensaje en un diagrama de secuencia corresponde a una operación en una clase, a un evento disparador, o a una transición en una máquina de estados.

Ejemplo:

Diagrama de Secuencia



Fuente: Rumbaugh, James; Jacobson Ivar y Booch GRady, 2000

2.11 Diagrama de colaboración:

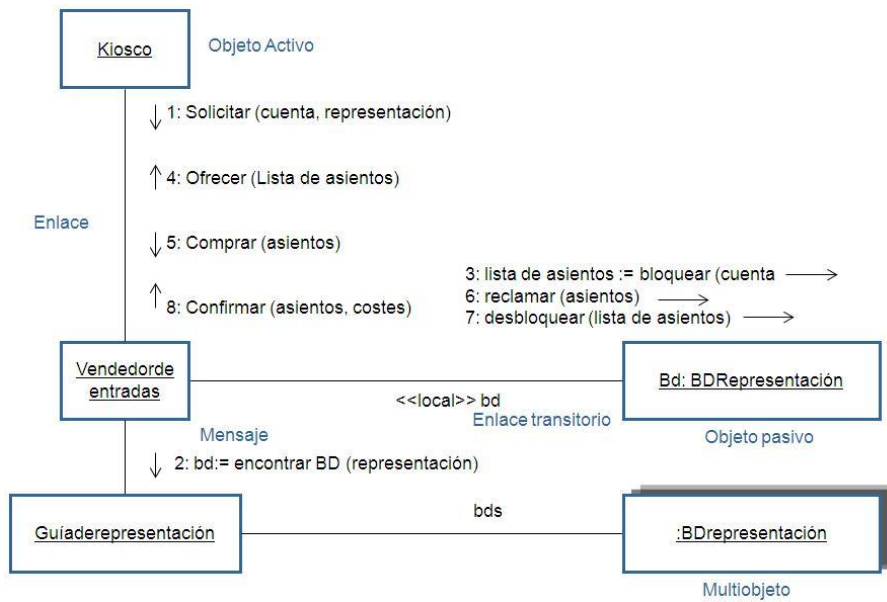
Es un diagrama de interacción que enfoca las interacciones y los enlaces entre un grupo de objetos "colaboradores". Ubicándose éste en el espacio mostrando los objetos, sus enlaces y los mensajes entre ellos. El Diagrama de Colaboración presenta una alternativa al diagrama de secuencia para modelar interacciones entre objetos en el sistema. Mientras que el diagrama de secuencia se centra en la secuencia cronológica del escenario que estamos modelando, el diagrama de colaboración se centra en estudiar todos los efectos de un objeto dado durante un escenario. Los objetos se conectan por medio de enlaces, cada enlace representa una instancia de una asociación entre las clases implicadas.

Características:

- Representa los efectos de un objeto dado durante un escenario.
- Los objetos se conectan por medio de enlaces, los enlaces representan una instancia de una asociación entre las clases implicadas.
- No muestra el tiempo como una dimensión aparte, por lo que se hace necesario etiquetar con números de secuencia tanto la secuencia de los mensajes como los hilos concurrentes.

Ejemplo:

Diagrama de Colaboración



2.12 Diagrama de tiempos:

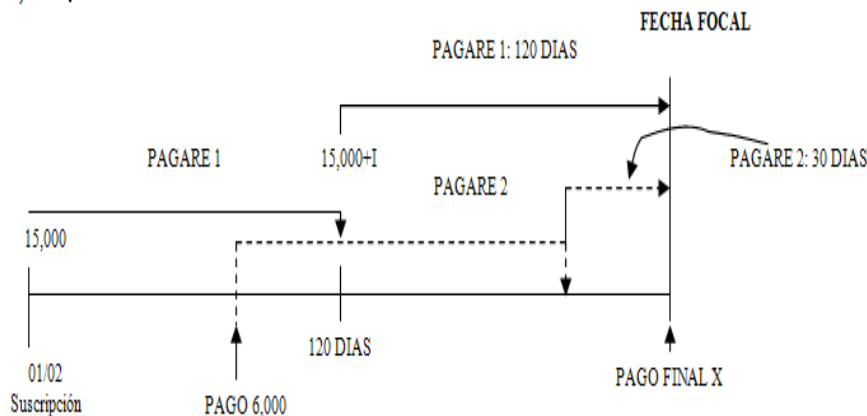
Los diagramas de tiempo son una representación especial de interacción que se enfoca en el tiempo de los mensajes enviados entre objetos. Se pueden usar estos diagramas para mostrar restricciones detalladas sobre el tiempo, ó para mostrar los cambios con líneas de vida respecto al tiempo.

Características:

- Los diagramas de tiempo son generalmente utilizados con sistemas en tiempo real o en sistemas embebidos.
- Examinando un diagrama de tiempos, se puede determinar los estados, nivel alto o nivel bajo, de cada una de las señales en cualquier instante de tiempo especificado, y el instante exacto en que cualquiera de las señales cambia de estado con respecto a las restantes.

Ejemplo:

Diagrama de Tiempo



Fuente: Ejemplo Diagrama de tiempo. MailxMail. Citado el 17/02/2009

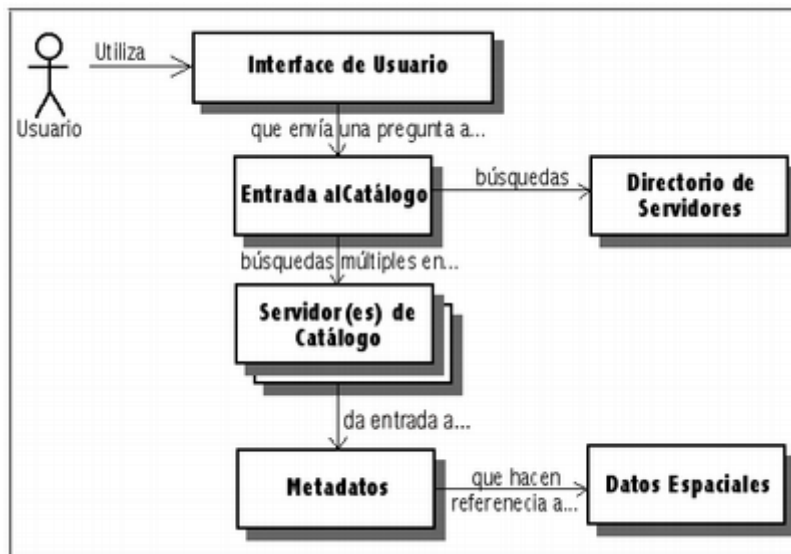
2.13 Diagrama de Interacción global: Muestran una interacción, que consiste de un conjunto de objetos y sus relaciones, incluyendo los mensajes que puedan ser realizados entre ellos. Son importantes para modelar los aspectos dinámicos de un sistema.

Características:

- Contienen objetos, enlaces y mensajes.
- Representan secuencia de intercambios y mensajes entre los roles que participan y se relacionan con un sistema.

Ejemplo:

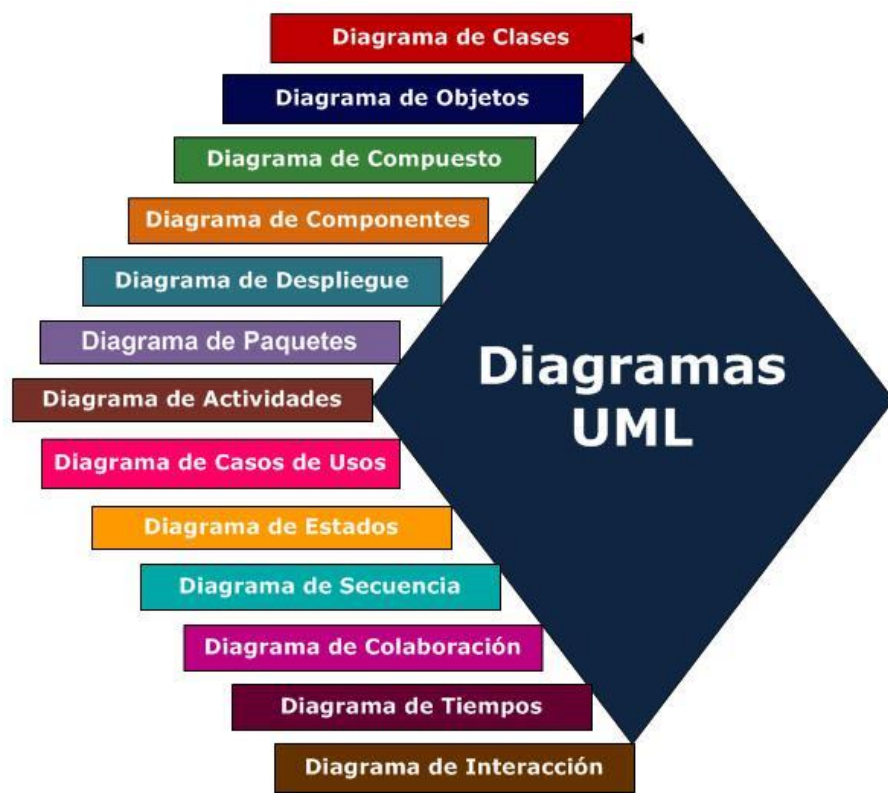
Diagrama de Interacción Global



Fuente: Diagrama de Interacción Global, [Interacción Global](#). Citado el 18/02/2009

3. Utilidad de cada uno de los diagramas.

Tomando en consideración que en todas las ramas de ingeniería se construyen modelos que permiten simplificar la realidad, para comprender el sistema que se va a desarrollar, UML utiliza trece diagramas para representar las distintas vistas de un sistema, los cuales se describen a continuación:



Fuente: Autores, Febrero 2009.

Diagramas de Estructura:

- **Diagramas de Clase:** representan una visión estática del sistema a desarrollar, es decir, permite describir las categorías con detalle de atributos de manera que el analista y el cliente puedan hablar el mismo lenguaje. Facilitan las representaciones sobre las cuales trabajan los desarrolladores. Presentan un conjunto de clases, interfaces y colaboraciones y las relaciones entre ellas. Su notación es un rectángulo dividido en tres áreas donde se especifica, nombre de la clase, atributos y acciones.
- **Diagrama de componentes:** permite mostrar la arquitectura física de un componente informático. Mostrando un conjunto de componentes y sus relaciones. Estos diagramas de componentes se utilizan para describir la vista de implementación estática de un sistema; se relacionan con los diagramas de clases, ya que un componente normalmente se corresponde con una o más clases, interfaces o colaboraciones.
- **Diagrama de objeto:** un objeto es una instancia de una clase, es decir, un objeto es aquel que representa valores específicos de los atributos y acciones definidos para la clase, por lo tanto este diagrama permite representar un objeto en UML. Estos diagramas son útiles para la comprensión de diagramas de clases completos, al crear diferentes casos en los que se aplican las relaciones y las clases. Se utiliza para describir estructuras de datos, instantáneas de los elementos encontrados en los diagramas de clases. Su notación es un rectángulo, con el nombre propio del objeto seguido de dos puntos (:) y el nombre de la clase a la cual pertenece.
- **Diagrama de estructura compuesta:** Los diagramas de estructura compuesta se usan para expresar arquitecturas en tiempos de ejecución, patrones de usos y las relaciones de todos los elementos participantes, los que pueden no estar reflejados por los diagramas estáticos.
- **Diagrama de despliegue:** Con este modelo se busca describir cómo la aplicación se despliega a través de una infraestructura.
- **Diagrama de paquetes:** Los diagramas de paquetes son utilizados para reflejar la organización de los paquetes y sus elementos, para así proveer una visualización de sus correspondientes nombres de espacio, los diagramas de paquete se asemejan mucho a la estructura del explorador de Windows.

Diagramas de Comportamiento:

- **Diagrama de actividades:** Las actividades de una secuencia se representan en este diagrama. Estos diagramas son el equivalente a los diagramas de flujo en la documentación clásica de software. Son importantes para modelar la función del sistema, así como para resaltar el flujo de control entre objetos.
- **Diagrama de casos de uso:** tomando en consideración la definición de casos de usos, los cuales son la descripción de las acciones del sistema desde el punto de vista del usuario, que permite a los desarrolladores obtener los requerimientos del sistema desde éste ángulo, se tiene que un diagrama de casos de uso organiza los comportamientos del sistema y representa un conjunto de casos de uso y actores (un tipo especial de clases) y sus relaciones. Muestra, por tanto, los distintos requisitos funcionales que se esperan de una aplicación o sistema y cómo se relacionan con su entorno (usuarios y otras aplicaciones).
- **Diagrama de estado:** Este diagrama resalta el comportamiento dirigido por eventos de un objeto, lo que es especialmente útil al modelar sistemas reactivos. En ellos se puede apreciar los estados que presenta un objeto en cualquier instante, ya que éste es dinámico; es decir, mientras que los diagramas de interacción muestran los objetos y los mensajes que se pasan entre ellos, un diagrama de estado muestra el estado cambiante de un solo objeto.

Diagramas de interacción:

- **Diagrama de tiempos:** con este diagrama se representan los cambios en el estado o la condición de una línea de vida a lo largo del tiempo lineal. Su principal uso es mostrar el cambio de estado de un objeto a lo largo del tiempo.
- **Diagrama de interacción, secuencia y colaboración:** Existen dos tipos de diagramas de interacción: los de secuencia y los de colaboración, estos diagramas representan la visión de comportamiento o interacción entre los diferentes componentes que conforman el sistema. Los diagramas de secuencia usan la perspectiva de la línea de tiempo para representar el comportamiento del sistema o componente de software, estas líneas se leen de izquierda a derecha del diagrama; mientras los diagramas de colaboración no están ordenados con relación al tiempo, por lo que su lectura está determinada por números que indican la secuencia de hechos.